

among the M abscissas $x_{j_{lo}}, \dots, x_{j_{lo}+M-1}$. By centered we mean that x lies between x_m and x_{m+1} insofar as possible, where

$$m = j_{lo} + \left\lfloor \frac{M-2}{2} \right\rfloor \quad (3.1.1)$$

By “insofar as possible” we mean that j_{lo} should never be less than zero, nor should $j_{lo} + M - 1$ be greater than $N - 1$.

In most cases, when all is said and done, it is hard to do better than *bisection*, which will find the right place in the table in about $\log_2 N$ tries.

`Int Base_interp::locate(const Doub x)`

`interp_1d.h`

Given a value x , return a value j such that x is (insofar as possible) centered in the subrange $xx[j..j+mm-1]$, where xx is the stored pointer. The values in xx must be monotonic, either increasing or decreasing. The returned value is not less than 0, nor greater than $n-1$.

```
{
    Int ju,jm,jl;
    if (n < 2 || mm < 2 || mm > n) throw("locate size error");
    Bool ascnd=(xx[n-1] >= xx[0]);      True if ascending order of table, false otherwise.
    jl=0;                               Initialize lower
    ju=n-1;                             and upper limits.
    while (ju-jl > 1) {                 If we are not yet done,
        jm = (ju+jl) >> 1;              compute a midpoint,
        if (x >= xx[jm] == ascnd)       and replace either the lower limit
            jl=jm;
        else
            ju=jm;                      or the upper limit, as appropriate.
    }                                   Repeat until the test condition is satisfied.
    cor = abs(jl-jsav) > dj ? 0 : 1;     Decide whether to use hunt or locate next time.
    jsav = jl;
    return MAX(0,MIN(n-mm,jl-((mm-2)>>1)));
}
```

The above `locate` routine accesses the array of values `xx[]` via a pointer stored by the base class. This rather primitive method of access, avoiding the use of a higher-level vector class like `VecDoub`, is here preferable for two reasons: (1) It's usually faster; and (2) for two-dimensional interpolation, we will later need to point directly into a row of a matrix. The peril of this design choice is that it assumes that consecutive values of a vector are stored consecutively, and similarly for consecutive values of a single row of a matrix. See discussion in §1.4.2.

3.1.1 Search with Correlated Values

Experience shows that in many, perhaps even most, applications, interpolation routines are called with nearly identical abscissas on consecutive searches. For example, you may be generating a function that is used on the right-hand side of a differential equation: Most differential equation integrators, as we shall see in Chapter 17, call for right-hand side evaluations at points that hop back and forth a bit, but whose trend moves slowly in the direction of the integration.

In such cases it is wasteful to do a full bisection, *ab initio*, on each call. Much more desirable is to give our base class a tiny bit of intelligence: If it sees two calls that are “close,” it anticipates that the next call will also be. Of course, there must not be too big a penalty if it anticipates wrongly.

The hunt method starts with a guessed position in the table. It first “hunts,” either up or down, in increments of 1, then 2, then 4, etc., until the desired value is bracketed. It then bisection in the bracketed interval. At worst, this routine is about a

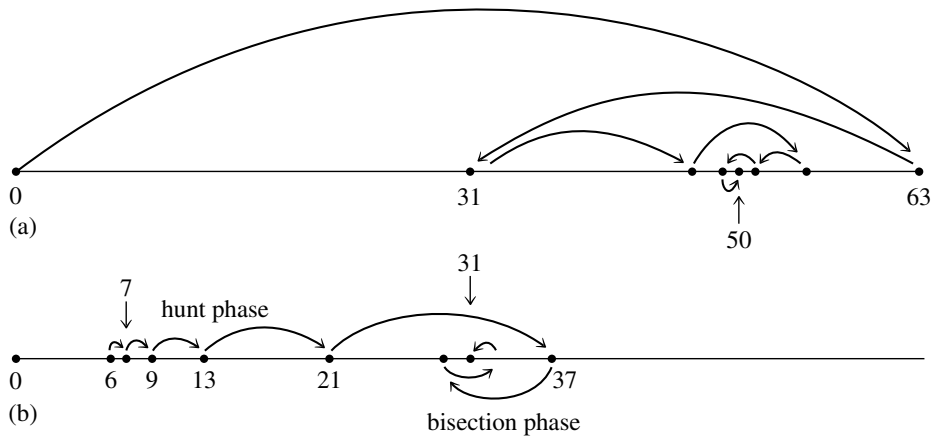


Figure 3.1.1. Finding a table entry by bisection. Shown here is the sequence of steps that converge to element 50 in a table of length 64. (b) The routine `hunt` searches from a previous known position in the table by increasing steps and then converges by bisection. Shown here is a particularly unfavorable example, converging to element 31 from element 6. A favorable example would be convergence to an element near 6, such as 8, which would require just three “hops.”

factor of 2 slower than `locate` above (if the hunt phase expands to include the whole table). At best, it can be a factor of $\log_2 n$ faster than `locate`, if the desired point is usually quite close to the input guess. Figure 3.1.1 compares the two routines.

`interp_1d.h`

`Int Base_interp::hunt(const Doub x)`

Given a value x , return a value j such that x is (insofar as possible) centered in the subrange $xx[j..j+mm-1]$, where xx is the stored pointer. The values in xx must be monotonic, either increasing or decreasing. The returned value is not less than 0, nor greater than $n-1$.

```
{
    Int jl=jsav, jm, ju, inc=1;
    if (n < 2 || mm < 2 || mm > n) throw("hunt size error");
    Bool ascnd=(xx[n-1] >= xx[0]);    True if ascending order of table, false otherwise.
    if (jl < 0 || jl > n-1) {         Input guess not useful. Go immediately to bisection.
        jl=0;
        ju=n-1;
    } else {
        if (x >= xx[jl] == ascnd) {   Hunt up:
            for (;;) {
                ju = jl + inc;
                if (ju >= n-1) { ju = n-1; break;}    Off end of table.
                else if (x < xx[ju] == ascnd) break;    Found bracket.
                else {                               Not done, so double the increment and try again.
                    jl = ju;
                    inc += inc;
                }
            }
        } else {                         Hunt down:
            ju = jl;
            for (;;) {
                jl = jl - inc;
                if (jl <= 0) { jl = 0; break;}    Off end of table.
                else if (x >= xx[jl] == ascnd) break;    Found bracket.
                else {                               Not done, so double the increment and try again.
                    ju = jl;
                    inc += inc;
                }
            }
        }
    }
}
```

```

        }
    }
}
while (ju-jl > 1) {
    jm = (ju+jl) >> 1;
    if (x >= xx[jm] == ascnd)
        jl=jm;
    else
        ju=jm;
}
cor = abs(jl-jsav) > dj ? 0 : 1;
jsav = jl;
return MAX(0,MIN(n-mm,jl-((mm-2)>>1)));
}

```

Hunt is done, so begin the final bisection phase:

Decide whether to use hunt or locate next time.

The methods `locate` and `hunt` each update the boolean variable `cor` in the base class, indicating whether consecutive calls seem correlated. That variable is then used by `interp` to decide whether to use `locate` or `hunt` on the next call. This is all invisible to the user, of course.

3.1.2 Example: Linear Interpolation

You may think that, at this point, we have wandered far from the subject of interpolation methods. To show that we are actually on track, here is a class that efficiently implements piecewise linear interpolation.

`struct Linear_interp : Base_interp`

`interp_linear.h`

Piecewise linear interpolation object. Construct with **x** and **y** vectors, then call `interp` for interpolated values.

```

{
    Linear_interp(VecDoub_I &xxv, VecDoub_I &yv)
        : Base_interp(xv,&yv[0],2) {}
    Doub rawinterp(Int j, Doub x) {
        if (xx[j]==xx[j+1]) return yy[j];
        else return yy[j] + ((x-xx[j])/(xx[j+1]-xx[j]))*(yy[j+1]-yy[j]);
    }
};

```

Table is defective, but we can recover.

You construct a linear interpolation object by declaring an instance with your filled vectors of abscissas x_i and function values $y_i = f(x_i)$,

```

Int n=...;
VecDoub xx(n), yy(n);
...
Linear_interp myfunc(xx,yy);

```

Behind the scenes, the base class constructor is called with $M = 2$ because linear interpolation uses just the two points bracketing a value. Also, pointers to the data are saved. (You must ensure that the vectors `xx` and `yy` don't go out of scope while `myfunc` is in use.)

When you want an interpolated value, it's as simple as

```

Doub x,y;
...
y = myfunc.interp(x);

```

If you have several functions that you want to interpolate, you declare a separate instance of `Linear_interp` for each one.

We will now use the same interface for more advanced interpolation methods.

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §6.2.1.

3.2 Polynomial Interpolation and Extrapolation

Through any two points there is a unique line. Through any three points there is a unique quadratic. *Et cetera*. The interpolating polynomial of degree $M - 1$ through the M points $y_0 = f(x_0), y_1 = f(x_1), \dots, y_{M-1} = f(x_{M-1})$ is given explicitly by Lagrange's classical formula,

$$\begin{aligned}
 P(x) = & \frac{(x - x_1)(x - x_2) \dots (x - x_{M-1})}{(x_0 - x_1)(x_0 - x_2) \dots (x_0 - x_{M-1})} y_0 \\
 & + \frac{(x - x_0)(x - x_2) \dots (x - x_{M-1})}{(x_1 - x_0)(x_1 - x_2) \dots (x_1 - x_{M-1})} y_1 + \dots \\
 & + \frac{(x - x_0)(x - x_1) \dots (x - x_{M-2})}{(x_{M-1} - x_0)(x_{M-1} - x_1) \dots (x_{M-1} - x_{M-2})} y_{M-1}
 \end{aligned} \tag{3.2.1}$$

There are M terms, each a polynomial of degree $M - 1$ and each constructed to be zero at all of the x_i 's except one, at which it is constructed to be y_i .

It is not terribly wrong to implement the Lagrange formula straightforwardly, but it is not terribly right either. The resulting algorithm gives no error estimate, and it is also somewhat awkward to program. A much better algorithm (for constructing the same, unique, interpolating polynomial) is *Neville's algorithm*, closely related to and sometimes confused with *Aitken's algorithm*, the latter now considered obsolete.

Let P_0 be the value at x of the unique polynomial of degree zero (i.e., a constant) passing through the point (x_0, y_0) ; so $P_0 = y_0$. Likewise define $P_1, P_2, \dots, P_{M-1}$. Now let P_{01} be the value at x of the unique polynomial of degree one passing through both (x_0, y_0) and (x_1, y_1) . Likewise $P_{12}, P_{23}, \dots, P_{(M-2)(M-1)}$. Similarly, for higher-order polynomials, up to $P_{012 \dots (M-1)}$, which is the value of the unique interpolating polynomial through all M points, i.e., the desired answer. The various P 's form a "tableau" with "ancestors" on the left leading to a single "descendant" at the extreme right. For example, with $M = 4$,

$$\begin{array}{rccccccc}
 x_0 : & y_0 = P_0 & & & & & \\
 & & P_{01} & & & & \\
 x_1 : & y_1 = P_1 & & P_{012} & & & \\
 & & P_{12} & & P_{0123} & & \\
 x_2 : & y_2 = P_2 & & P_{123} & & & \\
 & & P_{23} & & & & \\
 x_3 : & y_3 = P_3 & & & & &
 \end{array} \tag{3.2.2}$$

Neville's algorithm is a recursive way of filling in the numbers in the tableau a column at a time, from left to right. It is based on the relationship between a

“daughter” P and its two “parents,”

$$P_{i(i+1)\dots(i+m)} = \frac{(x - x_{i+m})P_{i(i+1)\dots(i+m-1)} + (x_i - x)P_{(i+1)(i+2)\dots(i+m)}}{x_i - x_{i+m}} \quad (3.2.3)$$

This recurrence works because the two parents already agree at points $x_{i+1} \dots x_{i+m-1}$.

An improvement on the recurrence (3.2.3) is to keep track of the small *differences* between parents and daughters, namely to define (for $m = 1, 2, \dots, M-1$),

$$\begin{aligned} C_{m,i} &\equiv P_{i\dots(i+m)} - P_{i\dots(i+m-1)} \\ D_{m,i} &\equiv P_{i\dots(i+m)} - P_{(i+1)\dots(i+m)}. \end{aligned} \quad (3.2.4)$$

Then one can easily derive from (3.2.3) the relations

$$\begin{aligned} D_{m+1,i} &= \frac{(x_{i+m+1} - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \\ C_{m+1,i} &= \frac{(x_i - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \end{aligned} \quad (3.2.5)$$

At each level m , the C ’s and D ’s are the corrections that make the interpolation one order higher. The final answer $P_{0\dots(M-1)}$ is equal to the sum of *any* y_i plus a set of C ’s and/or D ’s that form a path through the family tree to the rightmost daughter.

Here is the class implementing polynomial interpolation or extrapolation. All of its “support infrastructure” is in the base class `Base_interp`. It needs only to provide a `rawinterp` method that contains Neville’s algorithm.

`struct Poly_interp : Base_interp`

`interp_1d.h`

Polynomial interpolation object. Construct with **x** and **y** vectors, and the number M of points to be used locally (polynomial order plus one), then call `interp` for interpolated values.

```
{
    Doub dy;
    Poly_interp(VecDoub_I &xv, VecDoub_I &yv, Int m)
        : Base_interp(xv,&yv[0],m), dy(0.) {}
    Doub rawinterp(Int j1, Doub x);
};
```

`Doub Poly_interp::rawinterp(Int j1, Doub x)`

Given a value x , and using pointers to data xx and yy , this routine returns an interpolated value y , and stores an error estimate dy . The returned value is obtained by mm -point polynomial interpolation on the subrange $xx[j1..j1+mm-1]$.

```
{
    Int i,m,ns=0;
    Doub y,den,dif,dift,ho,hp,w;
    const Doub *xa = &xx[j1], *ya = &yy[j1];
    VecDoub c(mm),d(mm);
    dif=abs(x-xa[0]);
    for (i=0;i<mm;i++) {           Here we find the index ns of the closest table entry,
        if ((dift=abs(x-xa[i])) < dif) {
            ns=i;
            dif=dift;
        }
        c[i]=ya[i];                 and initialize the tableau of c's and d's.
        d[i]=ya[i];
    }
    y=ya[ns--];                    This is the initial approximation to y.
    for (m=1;m<mm;m++) {           For each column of the tableau,
```

```

for (i=0;i<mm-m;i++) {      we loop over the current c's and d's and update
    ho=xa[i]-x;              them.
    hp=xa[i+m]-x;
    w=c[i+1]-d[i];
    if ((den=ho-hp) == 0.0) throw("Poly_interp error");
    This error can occur only if two input xa's are (to within roundoff) identical.
    den=w/den;
    d[i]=hp*den;              Here the c's and d's are updated.
    c[i]=ho*den;
}
y += (dy=(2*(ns+1) < (mm-m) ? c[ns+1] : d[ns--]));
After each column in the tableau is completed, we decide which correction, c or d, we
want to add to our accumulating value of y, i.e., which path to take through the tableau
— forking up or down. We do this in such a way as to take the most "straight line"
route through the tableau to its apex, updating ns accordingly to keep track of where
we are. This route keeps the partial approximations centered (insofar as possible) on
the target x. The last dy added is thus the error indication.
}
return y;
}

```

The user interface to `Poly_interp` is virtually the same as for `Linear_interp` (end of §3.1), except that an additional argument in the constructor sets M , the number of points used (the order plus one). A cubic interpolator looks like this:

```

Int n=...;
VecDoub xx(n), yy(n);
...
Poly_interp myfunc(xx,yy,4);

```

`Poly_interp` stores an error estimate `dy` for the most recent call to its `interp` function:

```

Doub x,y,err;
...
y = myfunc.interp(x);
err = myfunc.dy;

```

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>, §25.2.
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.1.
- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.1.

3.3 Cubic Spline Interpolation

Given a tabulated function $y_i = y(x_i)$, $i = 0 \dots N - 1$, focus attention on one particular interval, between x_j and x_{j+1} . Linear interpolation in that interval gives the interpolation formula

$$y = Ay_j + By_{j+1} \quad (3.3.1)$$

where

$$A \equiv \frac{x_{j+1} - x}{x_{j+1} - x_j} \quad B \equiv 1 - A = \frac{x - x_j}{x_{j+1} - x_j} \quad (3.3.2)$$

Equations (3.3.1) and (3.3.2) are a special case of the general Lagrange interpolation formula (3.2.1).

Since it is (piecewise) linear, equation (3.3.1) has zero second derivative in the interior of each interval and an undefined, or infinite, second derivative at the abscissas x_j . The goal of cubic spline interpolation is to get an interpolation formula that is smooth in the first derivative and continuous in the second derivative, both within an interval and at its boundaries.

Suppose, contrary to fact, that in addition to the tabulated values of y_i , we also have tabulated values for the function's second derivatives, y'' , that is, a set of numbers y''_i . Then, within each interval, we can add to the right-hand side of equation (3.3.1) a cubic polynomial whose second derivative varies linearly from a value y''_j on the left to a value y''_{j+1} on the right. Doing so, we will have the desired continuous second derivative. If we also construct the cubic polynomial to have zero values at x_j and x_{j+1} , then adding it in will not spoil the agreement with the tabulated functional values y_j and y_{j+1} at the endpoints x_j and x_{j+1} .

A little side calculation shows that there is only one way to arrange this construction, namely replacing (3.3.1) by

$$y = Ay_j + By_{j+1} + Cy''_j + Dy''_{j+1} \quad (3.3.3)$$

where A and B are defined in (3.3.2) and

$$C \equiv \frac{1}{6}(A^3 - A)(x_{j+1} - x_j)^2 \quad D \equiv \frac{1}{6}(B^3 - B)(x_{j+1} - x_j)^2 \quad (3.3.4)$$

Notice that the dependence on the independent variable x in equations (3.3.3) and (3.3.4) is entirely through the linear x -dependence of A and B , and (through A and B) the cubic x -dependence of C and D .

We can readily check that y'' is in fact the second derivative of the new interpolating polynomial. We take derivatives of equation (3.3.3) with respect to x , using the definitions of A , B , C , and D to compute dA/dx , dB/dx , dC/dx , and dD/dx . The result is

$$\frac{dy}{dx} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{3A^2 - 1}{6}(x_{j+1} - x_j)y''_j + \frac{3B^2 - 1}{6}(x_{j+1} - x_j)y''_{j+1} \quad (3.3.5)$$

for the first derivative, and

$$\frac{d^2y}{dx^2} = Ay''_j + By''_{j+1} \quad (3.3.6)$$

for the second derivative. Since $A = 1$ at x_j , $A = 0$ at x_{j+1} , while B is just the other way around, (3.3.6) shows that y'' is just the tabulated second derivative, and also that the second derivative will be continuous across, e.g., the boundary between the two intervals (x_{j-1}, x_j) and (x_j, x_{j+1}) .

The only problem now is that we supposed the y''_i 's to be known, when, actually, they are not. However, we have not yet required that the *first* derivative, computed from equation (3.3.5), be continuous across the boundary between two intervals. The

key idea of a cubic spline is to require this continuity and to use it to get equations for the second derivatives y_i'' .

The required equations are obtained by setting equation (3.3.5) evaluated for $x = x_j$ in the interval (x_{j-1}, x_j) equal to the same equation evaluated for $x = x_j$ but in the interval (x_j, x_{j+1}) . With some rearrangement, this gives (for $j = 1, \dots, N-2$)

$$\frac{x_j - x_{j-1}}{6} y_{j-1}'' + \frac{x_{j+1} - x_{j-1}}{3} y_j'' + \frac{x_{j+1} - x_j}{6} y_{j+1}'' = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{y_j - y_{j-1}}{x_j - x_{j-1}} \quad (3.3.7)$$

These are $N-2$ linear equations in the N unknowns $y_i'', i = 0, \dots, N-1$. Therefore there is a two-parameter family of possible solutions.

For a unique solution, we need to specify two further conditions, typically taken as boundary conditions at x_0 and x_{N-1} . The most common ways of doing this are either

- set one or both of y_0'' and y_{N-1}'' equal to zero, giving the so-called *natural cubic spline*, which has zero second derivative on one or both of its boundaries, or
- set either of y_0'' and y_{N-1}'' to values calculated from equation (3.3.5) so as to make the first derivative of the interpolating function have a specified value on either or both boundaries.

Although the boundary condition for natural splines is commonly used, another possibility is to estimate the first derivatives at the endpoints from the first and last few tabulated points. For details of how to do this, see the end of §3.7. Best, of course, is if you can compute the endpoint first derivatives analytically.

One reason that cubic splines are especially practical is that the set of equations (3.3.7), along with the two additional boundary conditions, are not only linear, but also *tridiagonal*. Each y_j'' is coupled only to its nearest neighbors at $j \pm 1$. Therefore, the equations can be solved in $O(N)$ operations by the tridiagonal algorithm (§2.4). That algorithm is concise enough to build right into the function called by the constructor.

The object for cubic spline interpolation looks like this:

interp_1d.h

```
struct Spline_interp : Base_interp
Cubic spline interpolation object. Construct with x and y vectors, and (optionally) values of
the first derivative at the endpoints, then call interp for interpolated values.
{
    VecDoub y2;

    Spline_interp(VecDoub_I &xv, VecDoub_I &yv, Doub yp1=1.e99, Doub ypn=1.e99)
    : Base_interp(xv,yv,2), y2(xv.size())
    {sety2(&xv[0],&yv[0],yp1,ypn);}

    Spline_interp(VecDoub_I &xv, const Doub *yv, Doub yp1=1.e99, Doub ypn=1.e99)
    : Base_interp(xv,yv,2), y2(xv.size())
    {sety2(&xv[0],yv,yp1,ypn);}

    void sety2(const Doub *xv, const Doub *yv, Doub yp1, Doub ypn);
    Doub rawinterp(Int jl, Doub xv);
};
```

For now, you can ignore the second constructor; it will be used later for two-dimensional spline interpolation.

The user interface differs from previous ones only in the addition of two constructor arguments, used to set the values of the first derivatives at the endpoints, y'_0 and y'_{N-1} . These are coded with default values that signal that you want a natural spline, so they can be omitted in most situations. Both constructors invoke `sety2` to do the actual work of computing, and storing, the second derivatives.

```
void Spline_interp::sety2(const Doub *xv, const Doub *yv, Doub yp1, Doub ypn) interp_1d.h
This routine stores an array y2[0..n-1] with second derivatives of the interpolating function
at the tabulated points pointed to by xv, using function values pointed to by yv. If yp1 and/or
ypn are equal to  $1 \times 10^{99}$  or larger, the routine is signaled to set the corresponding boundary
condition for a natural spline, with zero second derivative on that boundary; otherwise, they are
the values of the first derivatives at the endpoints.
{
    Int i,k;
    Doub p,qn,sig,un;
    Int n=y2.size();
    VecDoub u(n-1);
    if (yp1 > 0.99e99) The lower boundary condition is set either to be "nat-
        y2[0]=u[0]=0.0; ural"
    else { or else to have a specified first derivative.
        y2[0] = -0.5;
        u[0]=(3.0/(xv[1]-xv[0]))*((yv[1]-yv[0])/(xv[1]-xv[0])-yp1);
    }
    for (i=1;i<n-1;i++) { This is the decomposition loop of the tridiagonal al-
        sig=(xv[i]-xv[i-1])/(xv[i+1]-xv[i-1]); gorithm. y2 and u are used for tem-
        p=sig*y2[i-1]+2.0; porary storage of the decomposed
        y2[i]=(sig-1.0)/p; factors.
        u[i]=(yv[i+1]-yv[i])/(xv[i+1]-xv[i]) - (yv[i]-yv[i-1])/(xv[i]-xv[i-1]);
        u[i]=(6.0*u[i]/(xv[i+1]-xv[i-1])-sig*u[i-1])/p;
    }
    if (ypn > 0.99e99) The upper boundary condition is set either to be
        qn=un=0.0; "natural"
    else { or else to have a specified first derivative.
        qn=0.5;
        un=(3.0/(xv[n-1]-xv[n-2]))*(ypn-(yv[n-1]-yv[n-2])/(xv[n-1]-xv[n-2]));
    }
    y2[n-1]=(un-qn*u[n-2])/(qn*y2[n-2]+1.0);
    for (k=n-2;k>=0;k--) This is the backsubstitution loop of the tridiagonal
        y2[k]=y2[k]*y2[k+1]+u[k]; algorithm.
}
```

Note that, unlike the previous object `Poly_interp`, `Spline_interp` stores data that depend on the contents of your array of y_i 's at its time of creation — a whole vector `y2`. Although we didn't point it out, the previous interpolation object actually allowed the misuse of altering the contents of their x and y arrays on the fly (as long as the lengths didn't change). If you do that with `Spline_interp`, you'll get definitely wrong answers!

The required `rawinterp` method, never called directly by the users, uses the stored `y2` and implements equation (3.3.3):

```
Doub Spline_interp::rawinterp(Int j1, Doub x) interp_1d.h
Given a value x, and using pointers to data xx and yy, and the stored vector of second derivatives
y2, this routine returns the cubic spline interpolated value y.
{
    Int klo=j1,khi=j1+1;
    Doub y,h,b,a;
    h=xx[khi]-xx[klo];
    if (h == 0.0) throw("Bad input to routine splint"); The xa's must be dis-
    a=(xx[khi]-x)/h; tinct.
```

```

b=(x-xx[klo])/h;                                Cubic spline polynomial is now evaluated.
y=a*yy[klo]+b*yy[khi]+((a*a*a-a)*y2[klo]
  +(b*b*b-b)*y2[khi])*(h*h)/6.0;
return y;
}

```

Typical use looks like this:

```

Int n=...;
VecDoub xx(n), yy(n);
...
Spline_interp myfunc(xx,yy);

```

and then, as often as you like,

```

Doub x,y;
...
y = myfunc.interp(x);

```

Note that no error estimate is available.

CITED REFERENCES AND FURTHER READING:

- De Boor, C. 1978, *A Practical Guide to Splines* (New York: Springer).
- Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, vol. 1 (Berlin: Springer), Chapter 9.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §4.4 – §4.5.
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.4.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §3.8.

3.4 Rational Function Interpolation and Extrapolation

Some functions are not well approximated by polynomials but *are* well approximated by rational functions, that is quotients of polynomials. We denote by $R_{i(i+1)\dots(i+m)}$ a rational function passing through the $m + 1$ points $(x_i, y_i), \dots, (x_{i+m}, y_{i+m})$. More explicitly, suppose

$$R_{i(i+1)\dots(i+m)} = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1x + \dots + p_\mu x^\mu}{q_0 + q_1x + \dots + q_\nu x^\nu} \quad (3.4.1)$$

Since there are $\mu + \nu + 1$ unknown p 's and q 's (q_0 being arbitrary), we must have

$$m + 1 = \mu + \nu + 1 \quad (3.4.2)$$

In specifying a rational function interpolating function, you must give the desired order of both the numerator and the denominator.

Rational functions are sometimes superior to polynomials, roughly speaking, because of their ability to model functions with poles, that is, zeros of the denominator of equation (3.4.1). These poles might occur for real values of x , if the function

to be interpolated itself has poles. More often, the function $f(x)$ is finite for all finite real x but has an analytic continuation with poles in the complex x -plane. Such poles can themselves ruin a polynomial approximation, even one restricted to real values of x , just as they can ruin the convergence of an infinite power series in x . If you draw a circle in the complex plane around your m tabulated points, then you should not expect polynomial interpolation to be good unless the nearest pole is rather far outside the circle. A rational function approximation, by contrast, will stay “good” as long as it has enough powers of x in its denominator to account for (cancel) any nearby poles.

For the interpolation problem, a rational function is constructed so as to go through a chosen set of tabulated functional values. However, we should also mention in passing that rational function approximations can be used in analytic work. One sometimes constructs a rational function approximation by the criterion that the rational function of equation (3.4.1) itself have a power series expansion that agrees with the first $m + 1$ terms of the power series expansion of the desired function $f(x)$. This is called *Padé approximation* and is discussed in §5.12.

Bulirsch and Stoer found an algorithm of the Neville type that performs rational function extrapolation on tabulated data. A tableau like that of equation (3.2.2) is constructed column by column, leading to a result and an error estimate. The Bulirsch-Stoer algorithm produces the so-called *diagonal* rational function, with the degrees of the numerator and denominator equal (if m is even) or with the degree of the denominator larger by one (if m is odd; cf. equation 3.4.2 above). For the derivation of the algorithm, refer to [1]. The algorithm is summarized by a recurrence relation exactly analogous to equation (3.2.3) for polynomial approximation:

$$R_{i(i+1)\dots(i+m)} = R_{(i+1)\dots(i+m)} + \frac{R_{(i+1)\dots(i+m)} - R_{i\dots(i+m-1)}}{\left(\frac{x-x_i}{x-x_{i+m}}\right) \left(1 - \frac{R_{(i+1)\dots(i+m)} - R_{i\dots(i+m-1)}}{R_{(i+1)\dots(i+m)} - R_{(i+1)\dots(i+m-1)}}\right) - 1} \quad (3.4.3)$$

This recurrence generates the rational functions through $m + 1$ points from the ones through m and (the term $R_{(i+1)\dots(i+m-1)}$ in equation 3.4.3) $m - 1$ points. It is started with

$$R_i = y_i \quad (3.4.4)$$

and with

$$R \equiv [R_{i(i+1)\dots(i+m)} \quad \text{with} \quad m = -1] = 0 \quad (3.4.5)$$

Now, exactly as in equations (3.2.4) and (3.2.5) above, we can convert the recurrence (3.4.3) to one involving only the small differences

$$\begin{aligned} C_{m,i} &\equiv R_{i\dots(i+m)} - R_{i\dots(i+m-1)} \\ D_{m,i} &\equiv R_{i\dots(i+m)} - R_{(i+1)\dots(i+m)} \end{aligned} \quad (3.4.6)$$

Note that these satisfy the relation

$$C_{m+1,i} - D_{m+1,i} = C_{m,i+1} - D_{m,i} \quad (3.4.7)$$